

PROJECT REPORT

Automated Web Application Deployment Using Ansible and NGINX Reverse Proxy

Prepared By: Mahadev

Date: 08-05-2026

Table of Contents

Contents

Table of Contents	2
1. Executive Summary	3
2. Introduction	3
3. Objectives	4
4. Technologies Used	5
5. System Architecture	5
5.1 Architecture Diagram	6
6. Project Implementation	6
6.1 Automated Tasks Performed	7
6.2 Deployment Workflow	7
6.3 Project Directory Structure	8
7. Application Configuration	11
8. Reverse Proxy Configuration	12
9. Security Configuration	14
10. Troubleshooting.....	16
11. Testing and Validation	18
12. Skills Learned.....	20
13. Future Improvements.....	21
14. Conclusion	22
15. References.....	22

1. Executive Summary

This project demonstrates the automated deployment of a Python Flask web application using Ansible automation and a NGINX reverse proxy on Red Hat Enterprise Linux systems.

The objective of the project was to simulate a practical DevOps deployment workflow by automating application setup, web server configuration, service management, and network communication between multiple Linux virtual machines.

The deployment environment consisted of two servers. The backend application server hosted the Flask application running on port 3000, while the frontend server functioned as an NGINX reverse proxy responsible for forwarding client requests to the backend service.

Ansible playbooks and roles were used to automate repetitive administrative tasks such as package installation, configuration deployment, firewall configuration, SELinux adjustments, and service management through systemd. Jinja2 templates were used to dynamically generate configuration files for both NGINX and systemd services.

During implementation, several real-world troubleshooting scenarios were encountered, including NGINX configuration conflicts, reverse proxy communication failures, firewall restrictions, and 502 Bad Gateway errors. These issues were analyzed and resolved using Linux networking and debugging tools.

The project successfully demonstrated core DevOps concepts including Infrastructure as Code (IaC), configuration management, automation, reverse proxy architecture, Linux service management, and deployment troubleshooting in a multi-server environment.

2. Introduction

Modern IT environments require fast, reliable, and consistent deployment methods. Manual server configuration and software deployment are time-consuming and can introduce configuration inconsistencies and operational errors.

This project focuses on automating the deployment of a Python Flask web application using Ansible automation on Red Hat Enterprise Linux systems. The implementation includes backend application deployment, NGINX reverse proxy configuration, firewall management, SELinux adjustments, and service management using systemd.

The project environment consisted of two Linux virtual machines configured in a multi-server architecture. One server acted as the backend application server hosting the Flask application on port 3000, while the second server acted as the frontend reverse proxy server using NGINX.

The deployment process was fully automated using Ansible playbooks, roles, tasks, templates, and handlers. Jinja2 templates were used to dynamically generate configuration files for services such as NGINX and systemd.

The project was designed to simulate a practical DevOps workflow and demonstrate Infrastructure as Code (IaC) principles, automation, Linux administration, and deployment troubleshooting in a controlled virtual environment.

3. Objectives

The primary objectives of this project were:

- Automate deployment of a Python Flask web application using Ansible
- Configure NGINX as a reverse proxy server
- Implement Infrastructure as Code (IaC) principles
- Manage Linux services using systemd
- Configure firewall and SELinux settings for secure communication
- Establish communication between frontend and backend servers
- Use Ansible roles, tasks, templates, and handlers for modular automation
- Simulate a real-world DevOps deployment workflow
- Troubleshoot deployment and network-related issues

4. Technologies Used

The following technologies were used to implement and automate the deployment environment.

Technology	Purpose
Ansible	Automation and configuration management
NGINX	Reverse proxy web server
Python Flask	Backend web application framework
systemd	Linux service management
Jinja2	Dynamic template generation
Firewalld	Firewall configuration
SELinux	Security policy enforcement
RHEL	Operating system environment

5. System Architecture

The project architecture consisted of two Linux virtual machines configured in a client-server deployment model.

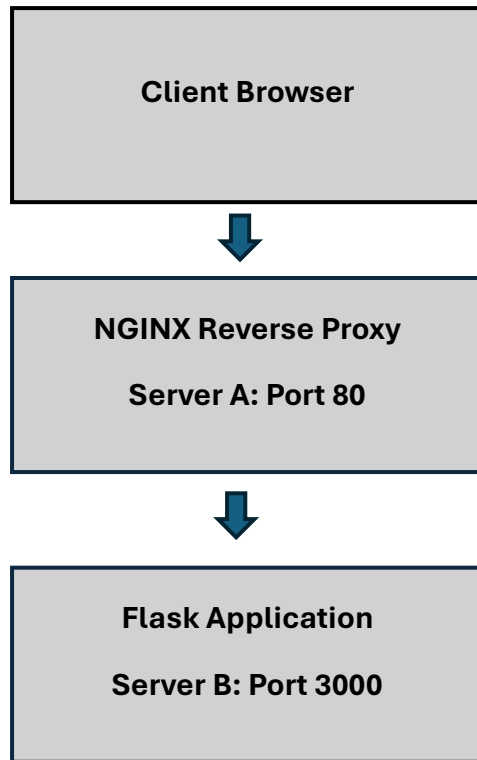
Server A functioned as the frontend server running NGINX as a reverse proxy. It accepted incoming HTTP requests from users on port 80 and forwarded them to the backend application server.

Server B functioned as the backend application server hosting a Python Flask web application running on port 3000. The Flask application processed client requests and returned responses through the NGINX reverse proxy.

Ansible automation was executed from the control node to configure both servers automatically. The playbooks handled package installation, application deployment, firewall configuration, SELinux adjustments, template deployment, and service management.

The deployment workflow simulated a practical DevOps infrastructure environment where frontend and backend services communicate securely across multiple Linux systems.

5.1 Architecture Diagram



communication flow between the client, reverse proxy server, and backend Flask application server.

6. Project Implementation

The implementation phase focused on automating the deployment and configuration of a Python Flask web application using Ansible automation tools on Red Hat Enterprise Linux virtual machines.

The project environment consisted of multiple Linux systems configured for frontend and backend communication. The backend server hosted the Flask application, while the frontend server was configured with NGINX reverse proxy services.

Ansible inventory files were used to organize managed hosts into groups. Playbooks and roles were created to automate administrative tasks across the deployment environment.

The automation workflow included package installation, Python dependency installation, application deployment, NGINX configuration, firewall rule management, SELinux configuration, and service management using systemd.

Jinja2 templates were used to dynamically generate configuration files such as the NGINX reverse proxy configuration and the systemd service file for the Flask application.

The Flask application was deployed on the backend server and configured to run continuously as a Linux service using systemd. The frontend NGINX server forwarded client requests to the backend application through reverse proxy configuration.

Firewall rules were configured to allow communication between frontend and backend systems. SELinux settings were also adjusted to allow NGINX reverse proxy communication with the backend application service.

The complete deployment process was automated through reusable Ansible playbooks, reducing manual configuration effort and improving deployment consistency across systems.

6.1 Automated Tasks Performed

- Package installation using Ansible package modules
- Python and Flask dependency installation
- Deployment of Flask application files
- NGINX reverse proxy configuration
- systemd service creation and management
- Firewall configuration using Firewalld
- SELinux policy adjustments
- Multi-server communication configuration
- Automated service restart handling
- Template deployment using Jinja2

6.2 Deployment Workflow

The deployment workflow was designed to automate the complete setup and configuration process of a multi-server web application environment using Ansible automation.

The process began from the Ansible control node, where inventory files were configured to define the managed hosts. Server A was configured as the frontend server running NGINX reverse proxy services, while Server B hosted the backend Python Flask application.

Ansible playbooks were executed to automate package installation, application deployment, firewall configuration, SELinux adjustments, template deployment, and service management across both servers.

The backend application deployment included Python installation, Flask dependency configuration, application file deployment, and creation of a systemd service to ensure automatic application startup and background execution.

The frontend server deployment configured NGINX as a reverse proxy server. Client HTTP requests received on port 80 were forwarded internally to the Flask application running on port 3000 on the backend server.

Firewall rules were configured to allow communication between servers, and SELinux policies were adjusted to permit NGINX reverse proxy communication with the backend application service.

Reusable Ansible roles, handlers, and Jinja2 templates were used to improve modularity, reduce repetition, and simplify configuration management.

During testing, multiple troubleshooting scenarios were encountered, including 502 Bad Gateway errors, backend connectivity failures, and firewall-related communication blocks. These issues were identified and resolved using Linux networking and debugging tools.

The automated deployment workflow successfully demonstrated Infrastructure as Code (IaC), configuration management, service orchestration, and real-world DevOps deployment practices in a Linux environment.

6.3 Project Directory Structure

The Ansible project was organized using a role-based directory structure to improve modularity, scalability, and maintainability. Separate roles were created for frontend web services, backend application deployment, and common system configuration tasks.

Figure 1: Ansible Role-Based Project Directory Structure

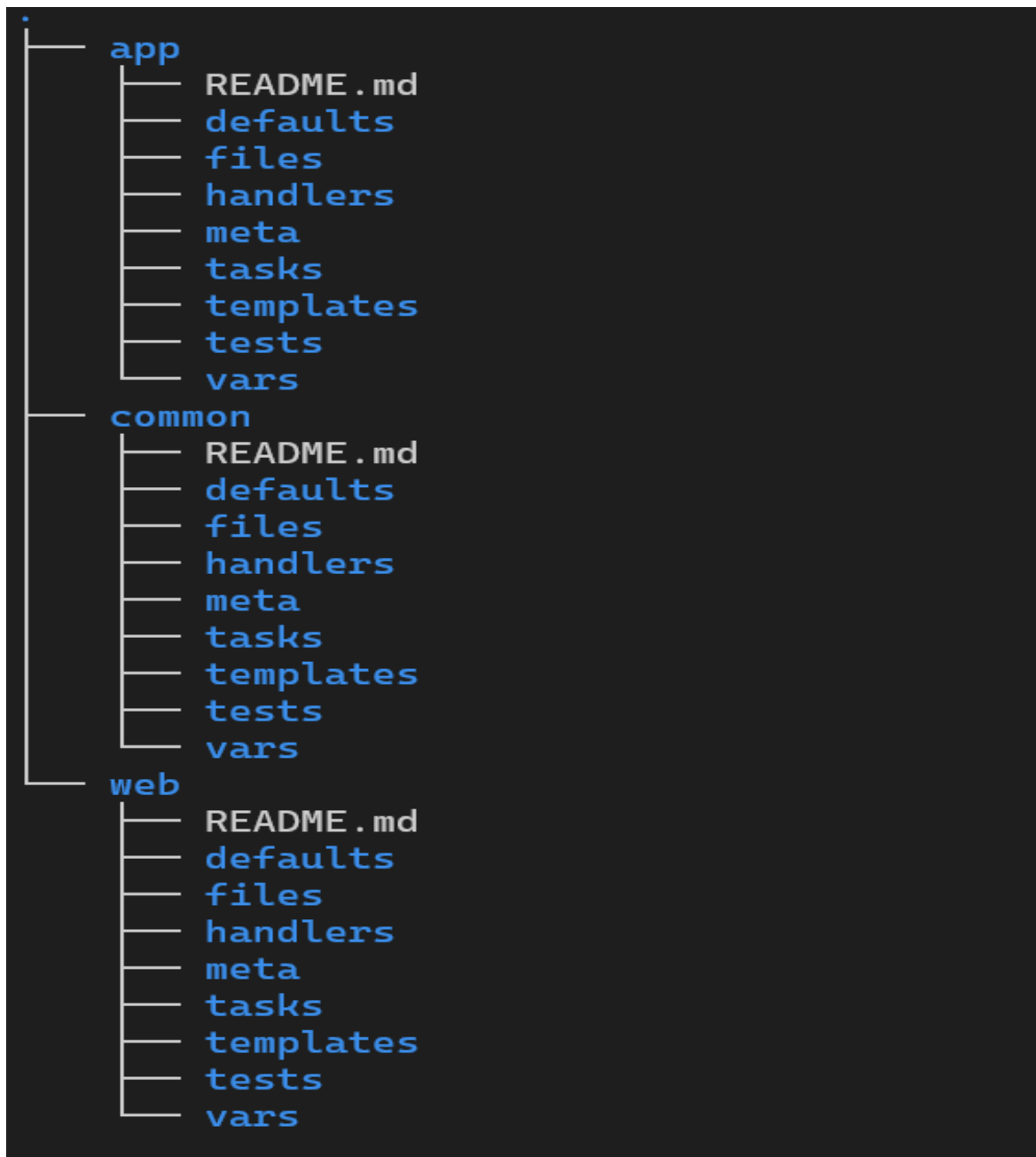


Figure 2: Flask Application Deployment Using Ansible Playbook

```
TASK [app : Install Flask] *****
ok: [serverb]

TASK [app : Create app directory] *****
ok: [serverb]

TASK [app : Copy application file] *****
ok: [serverb]

TASK [app : Create systemd service for app] *****
ok: [serverb]

TASK [app : Enable and start app] *****
ok: [serverb]

TASK [app : Allow Flask app port] *****
ok: [serverb]

PLAY [Configure webserver] *****

TASK [Gathering Facts] *****
ok: [servera]

TASK [web : install nginx] *****
```

Figure 3: NGINX Reverse Proxy Configuration and Successful Playbook Execution

```
workstation (workstation_ready) [Running] - Oracle VirtualBox

TASK [web : Clean nginx.conf (remove default server)] *****
ok: [servera]

TASK [web : Configure nginx reverse proxy] *****
ok: [servera]

TASK [web : Start and enable nginx] *****
ok: [servera]

TASK [web : Allow HTTP] *****
ok: [servera]

TASK [web : Allow nginx to connect to backend (SELinux)] *****
ok: [servera]

TASK [web : Add hosts entry] *****
ok: [servera]

PLAY RECAP *****
servera          : ok=14  changed=0    unreachable=0    failed=0    s
kipped=0         rescued=0    ignored=0
serverb          : ok=14  changed=0    unreachable=0    failed=0    s
kipped=0         rescued=0    ignored=0
```

7. Application Configuration

The backend application server was configured to host a Python Flask web application. The application environment was prepared using automated Ansible tasks to ensure consistent deployment across systems.

Python3 and required Flask dependencies were installed using Ansible package management modules. Application source files were automatically copied to the backend server through Ansible playbooks.

A dedicated systemd service was created to manage the Flask application process. The service configuration enabled automatic application startup during system boot and allowed centralized service management through Linux systemctl commands.

The Flask application was configured to run on port 3000 and listen for incoming requests forwarded from the NGINX reverse proxy server.

Application deployment tasks included:

- Python3 installation
- Flask dependency installation
- Application file deployment
- File permission configuration
- systemd service creation
- Service startup and enablement

Jinja2 templates were used to dynamically generate service configuration files, improving deployment flexibility and simplifying configuration management. The automated application configuration process reduced manual administrative effort and ensured deployment consistency across the backend environment.

Figure 4: Flask Application Service Status

```
dev@serverb ~]$ systemctl status myapp
■ myapp.service - Flask App
   Loaded: loaded (/etc/systemd/system/myapp.service; enabled; preset: disabled)
   Active: active (running) since Thu 2026-05-07 16:03:34 IST; 3h 31min ago
     Main PID: 4161 (python3)
        Tasks: 1 (limit: 35972)
      Memory: 20.1M
         CPU: 1.286s
       CGroup: /system.slice/myapp.service
              └─4161 /usr/bin/python3 /opt/app/app.py

May 07 16:03:45 serverb python3[4161]: * Running on http://127.0.0.1:3000
May 07 16:03:45 serverb python3[4161]: * Running on http://10.0.2.15:3000
May 07 16:03:45 serverb python3[4161]: Press CTRL+C to quit
May 07 16:04:18 serverb python3[4161]: 192.168.56.104 - - [07/May/2026 16:04:18>
May 07 16:04:21 serverb python3[4161]: 192.168.56.104 - - [07/May/2026 16:04:21>
May 07 16:04:22 serverb python3[4161]: 192.168.56.104 - - [07/May/2026 16:04:22>
May 07 16:04:22 serverb python3[4161]: 192.168.56.104 - - [07/May/2026 16:04:22>
May 07 16:04:22 serverb python3[4161]: 192.168.56.104 - - [07/May/2026 16:04:22>
May 07 16:04:22 serverb python3[4161]: 192.168.56.104 - - [07/May/2026 16:04:22>
May 07 16:04:23 serverb python3[4161]: 192.168.56.104 - - [07/May/2026 16:04:23>
```

Figure 5: Flask Application Response Verification

```
dev@serverb ~]$ curl http://localhost:3000
Hello! Visit count: 9dev@serverb ~]$ curl http://localhost:3000
Hello! Visit count: 10dev@serverb ~]$ curl http://localhost:3000
Hello! Visit count: 11dev@serverb ~]$ curl http://localhost:3000
Hello! Visit count: 12dev@serverb ~]$ curl http://localhost:3000
Hello! Visit count: 13dev@serverb ~]$ curl http://localhost:3000
Hello! Visit count: 14dev@serverb ~]$
```

8. Reverse Proxy Configuration

The frontend server was configured with NGINX to function as a reverse proxy server within the deployment architecture. The reverse proxy configuration allowed client requests to be forwarded securely to the backend Flask application running on the application server.

NGINX was configured to listen on port 80 for incoming HTTP requests. Requests received by the frontend server were internally forwarded to the backend Flask application running on port 3000.

The reverse proxy architecture improved deployment structure by separating frontend request handling from backend application processing. This configuration also prevented direct external access to the backend application service.

Ansible automation was used to deploy the NGINX configuration file using Jinja2 templates. This allowed configuration changes to be managed dynamically and consistently across deployments.

The NGINX configuration included reverse proxy directives and header forwarding configuration to preserve client request information during communication with the backend application.

The deployed reverse proxy configuration is shown below:

```
server {  
    listen 80 default_server;  
  
    server_name _;  
  
    location / {  
        proxy_pass http://192.168.56.105:3000;  
  
        proxy_set_header Host $host;  
        proxy_set_header X-Real-IP $remote_addr;  
    }  
}
```

The reverse proxy configuration was validated using nginx configuration tests and browser-based connectivity testing.

Reverse proxy deployment tasks included:

- NGINX package installation
- Reverse proxy configuration deployment
- Configuration validation using `nginx -t`
- HTTP service enablement
- NGINX service restart handling
- Backend connectivity validation

The successful reverse proxy implementation demonstrated frontend-backend communication and traffic forwarding within a multi-server Linux environment.

Figure 6: NGINX Reverse Proxy Service Status

```
[dev@servera ~]$ systemctl status nginx
■ nginx.service - The nginx HTTP and reverse proxy server
   Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; preset: disabled)
   Active: active (running) since Thu 2026-05-07 16:04:01 IST; 3h 35min ago
     Process: 4176 ExecStartPre=/usr/bin/rm -f /run/nginx.pid (code=exited, status=0/SUCCESS)
     Process: 4177 ExecStartPre=/usr/sbin/nginx -t (code=exited, status=0/SUCCESS)
     Process: 4178 ExecStart=/usr/sbin/nginx (code=exited, status=0/SUCCESS)
   Main PID: 4179 (nginx)
      Tasks: 5 (limit: 35972)
     Memory: 4.8M
        CPU: 35ms
    CGroup: /system.slice/nginx.service
            └─4179 "nginx: master process /usr/sbin/nginx"
              └─4180 "nginx: worker process"
                └─4181 "nginx: worker process"
                  └─4182 "nginx: worker process"
                    └─4183 "nginx: worker process"

May 07 16:04:01 servera systemd[1]: Starting The nginx HTTP and reverse proxy server:
May 07 16:04:01 servera nginx[4177]: nginx: the configuration file /etc/nginx/nginx.conf
May 07 16:04:01 servera nginx[4177]: nginx: configuration file /etc/nginx/nginx.conf
May 07 16:04:01 servera systemd[1]: Started The nginx HTTP and reverse proxy server:
```

Figure 7: Successful Reverse Proxy Communication

```
[dev@servera ~]$ curl http://localhost
Hello! Visit count: 15[dev@servera ~]$ curl http://localhost
Hello! Visit count: 16[dev@servera ~]$ curl http://localhost
Hello! Visit count: 17[dev@servera ~]$ curl http://localhost
Hello! Visit count: 18[dev@servera ~]$ curl http://localhost
Hello! Visit count: 19[dev@servera ~]$ curl http://localhost
Hello! Visit count: 20[dev@servera ~]$ curl http://localhost
Hello! Visit count: 21[dev@servera ~]$ curl http://localhost
```

9. Security Configuration

Security configuration was an important part of the deployment process because communication between the frontend NGINX server and the backend Flask application required proper firewall and SELinux configuration.

Firewalld was used to manage network access rules on the Linux servers. Required ports were opened to allow HTTP communication and backend application connectivity between systems.

The backend Flask application operated on port 3000, while the frontend NGINX reverse proxy accepted HTTP requests on port 80. Firewall rules were configured to permit communication through these ports.

Ansible automation was used to configure firewall settings consistently across systems using Firewalld modules.

Example firewall configuration task:

```
- name: Allow Flask application port
  ansible.posix.firewalld:
    port: 3000/tcp
    permanent: true
    state: enabled
    immediate: true
```

SELinux security policies initially restricted NGINX from communicating with the backend application service. As a result, reverse proxy requests failed even when firewall rules were configured correctly.

The issue was resolved by enabling the SELinux boolean:

```
httpd_can_network_connect
```

This configuration allowed NGINX processes to establish outbound network connections to backend application ports.

Example SELinux configuration task:

```
- name: Allow nginx backend communication
  ansible.posix.seboolean:
    name: httpd_can_network_connect
    state: true
    persistent: true
```

The security configuration process demonstrated the importance of Linux firewall management and SELinux policy handling in multi-server deployment environments.

Security configuration tasks included:

- Firewall port configuration
- HTTP service access configuration

- Backend application port access
- SELinux boolean configuration
- Reverse proxy communication validation
- Network connectivity testing

Figure 8: Firewall Configuration Verification

```
[dev@serverb ~]$ sudo firewall-cmd --list-all
public (active)
  target: default
  icmp-block-inversion: no
  interfaces: enp0s3 enp0s8
  sources:
  services: cockpit dhcpv6-client http ssh
  ports: 3000/tcp
  protocols:
  forward: yes
  masquerade: no
  forward-ports:
  source-ports:
  icmp-blocks:
  rich rules:
```

10. Troubleshooting

During implementation, several deployment and communication issues were encountered while configuring the multi-server application environment. These issues required troubleshooting of Linux services, network communication, firewall settings, SELinux policies, and reverse proxy configuration.

One of the primary issues encountered during deployment was the “502 Bad Gateway” error returned by NGINX. This error occurred because the reverse proxy server was unable to establish communication with the backend Flask application service.

Initial troubleshooting involved verifying the status of the Flask application service using `systemctl` commands and testing backend connectivity using `curl` requests.

The backend application was confirmed to be running correctly; however, communication between the frontend and backend servers was blocked due to firewall restrictions on port 3000.

Firewall rules were updated using `Firewalld` configuration commands to allow backend application traffic. After modifying firewall settings, backend connectivity improved but reverse proxy communication issues still persisted.

Further investigation revealed that SELinux policies were preventing NGINX from establishing outbound network connections to the backend application service.

The issue was resolved by enabling the SELinux boolean:

httpd_can_network_connect

After updating SELinux configuration, NGINX successfully forwarded client requests to the backend Flask application.

Additional troubleshooting scenarios included:

- Incorrect backend IP configuration in proxy_pass directives
- NGINX configuration conflicts caused by default server blocks
- Application service startup failures
- Configuration syntax validation errors
- Reverse proxy communication failures

Several Linux diagnostic and validation tools were used during troubleshooting, including:

- systemctl
- curl
- nginx -t
- firewall-cmd
- ss -tulnp
- journalctl

The troubleshooting process improved practical understanding of Linux networking, reverse proxy communication, service management, firewall configuration, and SELinux policy behaviour within a DevOps deployment environment.

Figure 9: Backend Application Connectivity Verification



```
[dev@servera ~]$ curl http://localhost
<html>
<head><title>502 Bad Gateway</title></head>
<body>
<center><h1>502 Bad Gateway</h1></center>
<hr><center>nginx/1.20.1</center>
</body>
</html>
```

11. Testing and Validation

After deployment and troubleshooting were completed, multiple validation procedures were performed to confirm successful operation of the automated web application environment.

Testing focused on verifying backend application availability, reverse proxy communication, service status, firewall configuration, and overall connectivity between frontend and backend systems.

The Flask backend application was first tested directly on the backend server using curl requests to confirm that the application service was running correctly on port 3000.

Example validation command:

```
curl http://localhost:3000
```

The frontend NGINX reverse proxy server was then tested to confirm successful forwarding of HTTP requests to the backend application service.

Example validation command:

```
curl http://localhost
```

Browser-based testing was also performed by accessing the frontend server IP address through a web browser. Successful page responses confirmed proper reverse proxy functionality and backend communication.

NGINX configuration syntax validation was performed using:

```
nginx -t
```

Service management validation was performed using:

```
systemctl status nginx
```

```
systemctl status myapp
```

Firewall configuration validation was performed using:

```
firewall-cmd --list-ports
```

SELinux configuration validation was performed using:

```
getsebool httpd_can_network_connect
```

The validation process confirmed:

- Successful deployment of the Flask application
- Proper operation of the NGINX reverse proxy
- Successful frontend-backend communication
- Correct firewall configuration
- Correct SELinux configuration
- Successful service management using systemd
- Successful Infrastructure as Code deployment using Ansible

The testing and validation phase confirmed that the automated deployment environment functioned correctly and met the project objectives.

Figure 10: Backend Flask Application Validation

```
[dev@serverb ~]$ curl http://localhost:3000
Hello! Visit count: 9[dev@serverb ~]$ curl http://localhost:3000
Hello! Visit count: 10[dev@serverb ~]$ curl http://localhost:3000
Hello! Visit count: 11[dev@serverb ~]$ curl http://localhost:3000
Hello! Visit count: 12[dev@serverb ~]$ curl http://localhost:3000
Hello! Visit count: 13[dev@serverb ~]$ curl http://localhost:3000
Hello! Visit count: 14[dev@serverb ~]$
```

Figure 11: Service Status Verification

```
[dev@servera ~]$ systemctl status nginx
■ nginx.service - The nginx HTTP and reverse proxy server
   Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; preset: d
   Active: active (running) since Thu 2026-05-07 16:04:01 IST; 3h 49min ago
   Process: 4176 ExecStartPre=/usr/bin/rm -f /run/nginx.pid (code=exited, sta
   Process: 4177 ExecStartPre=/usr/sbin/nginx -t (code=exited, status=0/SUCCE
   Process: 4178 ExecStart=/usr/sbin/nginx (code=exited, status=0/SUCCESS)
   Main PID: 4179 (nginx)
     Tasks: 5 (limit: 35972)
    Memory: 4.8M
       CPU: 41ms
    CGroup: /system.slice/nginx.service
           └─4179 "nginx: master process /usr/sbin/nginx"
             └─4180 "nginx: worker process"
               └─4181 "nginx: worker process"
                 └─4182 "nginx: worker process"
                   └─4183 "nginx: worker process"

May 07 16:04:01 servera systemd[1]: Starting The nginx HTTP and reverse proxy
May 07 16:04:01 servera nginx[4177]: nginx: the configuration file /etc/nginx/
May 07 16:04:01 servera nginx[4177]: nginx: configuration file /etc/nginx/ngin
May 07 16:04:01 servera systemd[1]: Started The nginx HTTP and reverse proxy s
```

12. Skills Learned

This project provided practical experience with several important DevOps, Linux administration, and automation concepts. The implementation process improved understanding of Infrastructure as Code principles, automated deployment workflows, reverse proxy architecture, and Linux troubleshooting techniques.

Through the deployment process, practical experience was gained in configuring Linux servers, automating repetitive administrative tasks, and managing multi-server application environments using Ansible automation tools.

The project also improved understanding of frontend-backend communication workflows and reverse proxy implementation using NGINX.

Important technical skills developed during the project included:

- Linux server administration
- Ansible automation and playbook development
- Role-based deployment structure
- Jinja2 template management
- Python Flask application deployment
- NGINX reverse proxy configuration
- systemd service management
- Firewalld configuration
- SELinux policy troubleshooting
- Network communication validation
- Deployment troubleshooting and debugging
- Infrastructure as Code (IaC) concepts

The troubleshooting phase of the project significantly improved practical debugging skills by requiring investigation of service failures, connectivity problems, firewall restrictions, SELinux policies, and configuration conflicts.

The project also strengthened understanding of structured deployment workflows commonly used in modern DevOps environments.

13. Future Improvements

Although the project successfully demonstrated automated deployment of a Flask web application using Ansible and NGINX, several additional improvements can be implemented to further enhance the deployment environment and introduce more advanced DevOps practices.

One possible enhancement is database integration using technologies such as PostgreSQL or MySQL. This would allow the application to store persistent data and simulate a more realistic production environment.

Containerization using Docker could also be implemented to simplify application deployment, improve portability, and standardize runtime environments across systems.

Future versions of the project may also include CI/CD pipeline integration using tools such as Jenkins or GitHub Actions. This would automate application testing and deployment whenever code changes are introduced.

Security improvements could include HTTPS implementation using SSL/TLS certificates to encrypt communication between clients and the reverse proxy server.

Cloud deployment on platforms such as AWS could further improve scalability and provide exposure to cloud infrastructure management concepts.

Additional monitoring and logging solutions could also be integrated using tools such as Prometheus, Grafana, or centralized logging systems to improve system visibility and operational monitoring.

Possible future improvements include:

- Database integration using PostgreSQL or MySQL
- Docker containerization
- CI/CD pipeline implementation
- HTTPS configuration using SSL certificates
- Cloud deployment on AWS
- Monitoring and logging integration
- Load balancing configuration
- Multi-application deployment support

These improvements would extend the project into a more advanced production-style DevOps deployment environment.

14. Conclusion

This project successfully demonstrated the automated deployment of a Python Flask web application using Ansible automation and NGINX reverse proxy services on Red Hat Enterprise Linux systems.

The implementation covered multiple practical DevOps concepts including Infrastructure as Code (IaC), automated configuration management, reverse proxy deployment, Linux service management, firewall configuration, SELinux policy handling, and deployment troubleshooting.

The deployment environment consisted of separate frontend and backend Linux servers communicating through a reverse proxy architecture. Ansible playbooks, roles, handlers, and Jinja2 templates were used to automate package installation, application deployment, configuration management, and service orchestration.

During implementation, several real-world troubleshooting scenarios were encountered, including reverse proxy communication failures, firewall restrictions, SELinux policy conflicts, and NGINX configuration issues. Resolving these problems improved practical understanding of Linux administration and deployment debugging techniques.

The project successfully validated frontend-backend communication, reverse proxy functionality, service management, and Infrastructure as Code deployment practices within a multi-server Linux environment.

Overall, the project provided valuable hands-on experience with automation, Linux system administration, DevOps workflows, and deployment troubleshooting techniques commonly used in modern infrastructure environments.

15. References

- Ansible Official Documentation
<https://docs.ansible.com/>
- NGINX Official Documentation
<https://nginx.org/en/docs/>

- Flask Official Documentation
<https://flask.palletsprojects.com/>
- Red Hat Enterprise Linux Documentation
<https://access.redhat.com/documentation/>
- FirewallD Documentation
<https://firewalld.org/documentation/>
- SELinux Project Documentation
<https://selinuxproject.org/>
- Community troubleshooting discussions from Reddit, GitHub, and DevOps forums
- Linux manual pages and system administration references